The Journal of Open Source Software

Simulation-based Inference with the Python Package sbijax

Simon Dirmeier[1,2], **Antonietta Mira**[3,4], **and Carlo Albert**[5]

1 Swiss Data Science Center, Zurich, Switzerland **2** ETH Zurich, Zurich, Switzerland **3** Università della Svizzera italiana, Switzerland **4** University of Insubria, Italy **5** Swiss Federal Institute of Aquatic Science and Technology, Switzerland

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor: Richard Littauer

Reviewers:

- @aneeshnaik
- @dosvk

Submitted: 19 March 2026
Published: unpublished

Summary

Neural simulation-based inference (SBI) describes an emerging family of methods for Bayesian inference for simulator models that use neural networks as surrogate models. Here we introduce `sbijax`, a Python package that implements a wide variety of state-of-the-art methods in neural simulation-based inference using a user-friendly programming interface. sbijax offers high-level functionality to quickly construct SBI estimators, and compute and visualize posterior distributions with only a few lines of code. In addition, the package provides functionality for conventional approximate Bayesian computation, to compute model diagnostics, and to automatically estimate summary statistics. By virtue of being entirely written in `JAX`, sbijax is extremely computationally efficient, allowing rapid training of neural networks and executing code automatically in parallel on both CPU and GPU.

Statement of Need

Modern approaches to neural simulation-based inference (SBI) utilize recent developments in neural density estimation or score-based generative modelling to build surrogate models to approximate Bayesian posterior distributions. Similarly to conventional methods, such as approximate Bayesian computation (ABC) and its sequential (SMC-ABC) and annealing-based (e.g., SABC) variants, neural SBI methods infer this posterior distribution by first simulating synthetic data and then numerically constructing an appropriate approximation to this pseudo data set. SBI methods are attractive for a couple of reasons. On the one hand, this family of methods has been shown to be more computationally efficient and often more accurate than ABC methods, in particular for smaller simulation budgets. On the other hand, SBI allows to easily amortize inference, i.e., to infer the posterior distribution for multiple different observations once a neural model has been trained.

Here we propose `sbijax`, a Python package implementing state-of-the-art methodology of neural simulation-based inference. While the main focus of the package is the implementation of recent algorithms to make them available to practitioners, e.g., Wildberger et al. (2023) or Schmitt et al. (2023), `sbijax` also implements common methods from approximate Bayesian computation, e.g., SMC-ABC (Beaumont et al., 2009), to have the entire SBI toolbox in one efficient package (see Table 1 for an overview). In addition, `sbijax` provides functionality for model diagnostics, posterior visualization and Markov Chain Monte Carlo (MCMC) sampling.

The package uses the high-performance computing framework `JAX` as a backend (Bradbury et al., 2018). Using `JAX` has several advantages, including a) that it uses the same syntax as `numpy` (Harris et al., 2020) which enables a seamless transition for applied scientists who already are familiar with it, and b) that empirical evaluations have shown that `JAX` can be significantly faster than `PyTorch` (see, e.g., Phan et al. (2019)). Our package heavily builds on

libraries from the JAX-verse and common Bayesian inference tools. Specifically, we use `Haiku` (Hennigan et al., 2020) to construct and train neural networks, `ArviZ` (Kumar et al., 2019) for various model visualizations, `surjectors` for normalizing-flow based density estimation (Dirmeier, 2024), TensorFlow Probability (Dillon et al., 2017) to define statistical distributions, and `BlackJAX` (Cabezas et al., 2024) for posterior sampling using Markov Chain Monte Carlo.

Table 1: Implemented SBI methods in `sbijax` .

Model	Class name	Reference
Sequential Monte Carlo ABC	`SMCABC`	Beaumont et al. (2009)
Neural likelihood estimation	`NLE`	Papamakarios et al. (2019)
Surjective neural likelihood estimation	`SNLE`	Dirmeier et al. (2025)
Automatic posterior transformation	`NPE`	Greenberg et al. (2019)
Contrastive neural ratio estimation	`NRE`	Miller et al. (2022)
Flow matching posterior estimation	`FMPE`	Wildberger et al. (2023)
Posterior Score Estimation	`NPSE`	Sharrock et al. (2024)
All-In-One Posterior Estimation	`AIO`	Gloeckler et al. (2024)
Consistency model posterior estimation	`CMPE`	Schmitt et al. (2023)
Neural approximate sufficient statistics	`NASS`	Chen et al. (2021)
Neural approximate slice sufficient statistics	`NASSS`	Chen et al. (2023)

State of the field

While a plethora of different models has been proposed in the recent literature, the development of adequate software packages has not followed at the same pace, and only few packages exist that allow modelers to use these methods. Most prominently, the Python package `sbi` (Tejero-Cantero et al., 2020) implements several approaches for neural simulation-based inference, such as a neural posterior, likelihood-ratio, and likelihood estimation (Cranmer et al., 2020) utilizing a `PyTorch` backend (Paszke et al., 2019). The package additionally provides an API for model diagnostics, such as posterior predictive checks, effective sample size computations and simulation-based calibration. However, the package lacks implementations of recent developments which pose the state-of-the-art in the field, such as by Chen et al. (2023), Dirmeier et al. (2025) or Schmitt et al. (2023). Also, by virtue of being developed in `PyTorch` it is potentially restrictive to practitioners that do not have experience with it. For approximate Bayesian computation, several `Python` packages are available. In particular `abcpy` (Dutta et al., 2021) implements a multitude of different ABC algorithms. However, none of these packages implement modern (neural) SBI methods.

Software design

`Sbijax` is designed as a modular and extensible toolbox for SBI in `JAX`, combining a functional programming philosophy with a flexible interface for both expert users and domain scientists. Its design is guided by the following principles:

1) Faithful alignment with `JAX`'s functional paradigm. Sbijax adheres closely to the low-level, functional programming model of `JAX`. While SBI methods are implemented using an object-oriented structure, all member functions return immutable objects rather than modifying internal state. This avoids hidden side effects, facilitates composability, and ensures compatibility with `JAX` transformations such as `jit`, `vmap`, and `pmap`.

2) Separation of inference methods and neural network implementations. Sbijax focuses on implementing SBI algorithms, while neural network components are defined using `Haiku`. This allows users to construct models directly with `Haiku`, and to incorporate

probabilistic building blocks from `Distrax` (DeepMind et al., 2020) and `surjectors` (Dirmeier, 2024). As a result, model definitions remain flexible and low-level, while seamlessly interoperating with the broader `JAX` ecosystem.

3) Support for extensibility and research. Sbijax is structured to facilitate experimentation with new SBI methods. Its modular design allows components such as neural architectures, training objectives, and sampling strategies to be easily replaced or extended, making it suitable as both a research framework and a practical toolbox.

4) Accessibility for domain scientists. In addition to its flexibility, Sbijax includes pre-implemented models with sensible defaults, enabling use without in-depth expertise in deep learning. In the simplest case, users only need to define a prior and a simulator to run inference workflows—often in as few as five lines of code.

Research impact statement

`sbijax` has already been used extensively in the Machine Learning literature. Dirmeier et al. (2025) and Dirmeier & Mira (2025) proposed novel SBI methods for surjective neural likelihood estimation and posterior estimation using causal constraints, respectively, where they used `sbijax` heavily for their experimental evaluations. Ulzega et al. (2025) used `sbijax` to infer the posterior distribution of a complicated Bayesian model from the astrophysics literature. Albert et al. (2025) developed a novel ABC method that uses `sbijax` for model evaluation.

AI usage disclosure

No GenAI or other AI tools have been used in writing the software or this manuscript.

Acknowledgements

This research was supported by the Swiss National Science Foundation (Grant No. 200021_208249).

References

Albert, C., Ulzega, S., Dirmeier, S., Scheidegger, A., Bassi, A., & Mira, A. (2025). Simulated annealing ABC with multiple summary statistics. *arXiv Preprint arXiv:2505.23261*.

Beaumont, M. A., Cornuet, J.-M., Marin, J.-M., & Robert, C. P. (2009). Adaptive approximate bayesian computation. *Biometrika*, *96*(4), 983–990.

Bradbury, J., Frostig, R., Hawkins, P., Johnson, M. J., Leary, C., Maclaurin, D., Necula, G., Paszke, A., VanderPlas, J., Wanderman-Milne, S., & Zhang, Q. (2018). *JAX: Composable transformations of Python+NumPy programs* (Version 0.3.13). http://github.com/google/jax

Cabezas, A., Corenflos, A., Lao, J., & Louf, R. (2024). BlackJAX: Composable Bayesian inference in JAX. *arXiv Preprint arXiv:2402.10797*.

Chen, Y., Gutmann, M. U., & Weller, A. (2023). Is learning summary statistics necessary for likelihood-free inference? *Proceedings of the 40th International Conference on Machine Learning*.

Chen, Y., Zhang, D., Gutmann, M. U., Courville, A., & Zhu, Z. (2021). Neural approximate sufficient statistics for implicit models. *International Conference on Learning Representations*.

Cranmer, K., Brehmer, J., & Louppe, G. (2020). The frontier of simulation-based inference. *Proceedings of the National Academy of Sciences*, *117*(48), 30055–30062.

DeepMind, Babuschkin, I., Baumli, K., Bell, A., Bhupatiraju, S., Bruce, J., Buchlovsky, P., Budden, D., Cai, T., Clark, A., Danihelka, I., Dedieu, A., Fantacci, C., Godwin, J., Jones, C., Hemsley, R., Hennigan, T., Hessel, M., Hou, S., … Viola, F. (2020). *The DeepMind JAX Ecosystem*. http://github.com/deepmind

Dillon, J. V., Langmore, I., Tran, D., Brevdo, E., Vasudevan, S., Moore, D., Patton, B., Alemi, A., Hoffman, M., & Saurous, R. A. (2017). Tensorflow distributions. *arXiv Preprint arXiv:1711.10604*.

Dirmeier, S. (2024). Surjectors: Surjection layers for density estimation with normalizing flows. *Journal of Open Source Software*, *9*(94), 6188.

Dirmeier, S., Albert, C., & Perez-Cruz, F. (2025). Simulation-based inference for high-dimensional data using surjective sequential neural likelihood estimation. *The 41st Conference on Uncertainty in Artificial Intelligence*.

Dirmeier, S., & Mira, A. (2025). Causal posterior estimation. *arXiv Preprint arXiv:2505.21468*.

Dutta, R., Schoengens, M., Pacchiardi, L., Ummadisingu, A., Widmer, N., Künzli, P., Onnela, J.-P., & Mira, A. (2021). ABCpy: A high-performance computing perspective to approximate bayesian computation. *Journal of Statistical Software*, *100*(7), 1–38.

Gloeckler, M., Deistler, M., Weilbach, C. D., Wood, F., & Macke, J. H. (2024). All-in-one simulation-based inference. *Proceedings of the 41st International Conference on Machine Learning*.

Greenberg, D., Nonnenmacher, M., & Macke, J. (2019). Automatic posterior transformation for likelihood-free inference. *Proceedings of the 36th International Conference on Machine Learning*.

Harris, C. R., Millman, K. J., Van Der Walt, S. J., Gommers, R., Virtanen, P., Cournapeau, D., Wieser, E., Taylor, J., Berg, S., Smith, N. J., & others. (2020). Array programming with NumPy. *Nature*, *585*(7825), 357–362.

Hennigan, T., Cai, T., Norman, T., Martens, L., & Babuschkin, I. (2020). *Haiku: Sonnet for JAX* (Version 0.0.10). http://github.com/deepmind/dm-haiku

Kumar, R., Carroll, C., Hartikainen, A., & Martin, O. (2019). ArviZ a unified library for exploratory analysis of bayesian models in python. *Journal of Open Source Software*, *4*(33), 1143.

Miller, B. K., Weniger, C., & Forré, P. (2022). Contrastive neural ratio estimation. *Advances in Neural Information Processing Systems*.

Papamakarios, G., Sterratt, D., & Murray, I. (2019). Sequential neural likelihood: Fast likelihood-free inference with autoregressive flows. *Proceedings of the 22nd International Conference on Artificial Intelligence and Statistics*.

Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., Desmaison, A., Kopf, A., Yang, E., DeVito, Z., Raison, M., Tejani, A., Chilamkurthy, S., Steiner, B., Fang, L., … Chintala, S. (2019). PyTorch: An imperative style, high-performance deep learning library. *Advances in Neural Information Processing Systems*.

Phan, D., Pradhan, N., & Jankowiak, M. (2019). Composable effects for flexible and accelerated probabilistic programming in NumPyro. *arXiv Preprint arXiv:1912.11554*.

Schmitt, M., Pratz, V., Köthe, U., Bürkner, P.-C., & Radev, S. T. (2023). Consistency models for scalable and fast simulation-based inference. *arXiv Preprint arXiv:2312.05440*.

Sharrock, L., Simons, J., Liu, S., & Beaumont, M. (2024). Sequential neural score estimation: Likelihood-free inference with conditional score based diffusion models. *Forty-First International Conference on Machine Learning*.

Tejero-Cantero, A., Boelts, J., Deistler, M., Lueckmann, J.-M., Durkan, C., Gonçalves, P. J., Greenberg, D. S., & Macke, J. H. (2020). Sbi: A toolkit for simulation-based inference. *Journal of Open Source Software*, *5*(52), 2505.

Ulzega, S., Beer, J., Ferriz-Mas, A., Dirmeier, S., & Albert, C. (2025). Shedding light on the solar dynamo using data-driven bayesian parameter inference. *The Astrophysical Journal*, *992*(1), 61.

Wildberger, J. B., Dax, M., Buchholz, S., Green, S. R., Macke, J. H., & Schölkopf, B. (2023). Flow matching for scalable simulation-based inference. *Advances in Neural Information Processing Systems*.